

TD4 Calcul Parallèle

Écrire la fonction qui permet de réduire le bruit (sur la moitié gauche).
Pour un effet plus important, il faut augmenter la taille du masque :

```
Entrée [23]: import numpy as np
from scipy.signal import convolve2d

def convolution(image, kernel):
    # Assurez-vous que la taille du noyau est impaire
    assert kernel.shape[0] % 2 == 1 and kernel.shape[1] % 2 == 1, "La taille du noyau doit être impaire"

    # Appliquer la convolution
    result1 = convolve2d(image, kernel, mode='same', boundary='symm')

    return result1

def reduce_noise(image):
    # Définir un masque de convolution pour le filtre moyenneur
    kernel = np.ones((5, 5)) / 25.0 # Pour un filtre 5x5

    # Appliquer la convolution sur la moitié gauche de l'image
    result = image.copy()
    result[:, :result.shape[1] // 2] = convolution(result[:, :result.shape[1] // 2], kernel)

    return result

# Charger l'image
image_path = "v1.jpg"
original_image = np.array(Image.open(image_path).convert("L"))

# Réduire le bruit sur la moitié gauche
image_with_noise_reduction = reduce_noise(original_image)

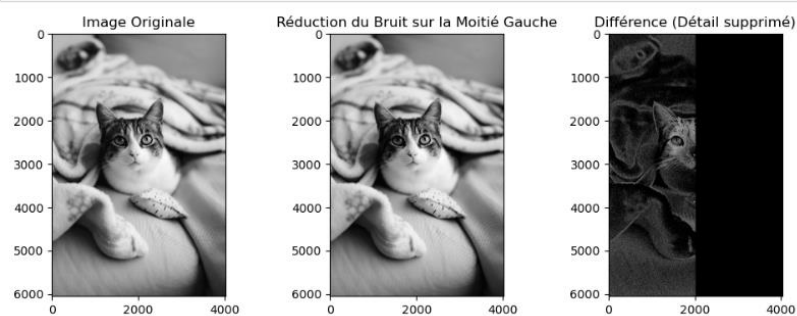
# Afficher les images d'origine et résultante
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(original_image, cmap='gray')
plt.title('Image Originale')

plt.subplot(1, 3, 2)
plt.imshow(image_with_noise_reduction, cmap='gray')
plt.title('Réduction du Bruit sur la Moitié Gauche')

plt.subplot(1, 3, 3)
plt.imshow(original_image - image_with_noise_reduction, cmap='gray')
plt.title('Différence (Détail supprimé)')

plt.show()
```



Écrire la fonction qui permet de créer un filtre gaussien. La matrice est normalisée pour que la somme de ses coefficients soit égale à 1. Tester l'effet d'un filtre gaussien 7x7 sur l'image bruitée: Comparer au filtre moyennneur de taille 5x5.

```
Entrée [22]: import numpy as np
from scipy.signal import convolve2d
import matplotlib.pyplot as plt
from PIL import Image

def gaussian_filter(size, sigma):
    """Créer un filtre gaussien."""
    kernel = np.fromfunction(
        lambda x, y: (1/(2*np.pi*sigma**2)) * np.exp(-((x - size//2)**2 + (y - size//2)**2)/(2*sigma**2)),
        (size, size)
    )
    return kernel / np.sum(kernel)

def apply_filter(image, kernel):
    """Appliquer un filtre à une image."""
    return convolve2d(image, kernel, mode='same', boundary='symm')

# Charger une image en niveaux de gris (par exemple, utiliser Pillow)
image_path = "v1.jpg"
original_image = np.array(Image.open(image_path).convert("L"))

# Ajouter du bruit à l'image (à des fins de démonstration)
image_noisy = original_image + np.random.normal(0, 25, original_image.shape)

# Créer un filtre gaussien 7x7
gaussian_kernel = gaussian_filter(7, 1)

# Appliquer le filtre gaussien à l'image bruitée
image_filtered_gaussian = apply_filter(image_noisy, gaussian_kernel)

# Créer un filtre moyennneur 5x5
mean_kernel = np.ones((5, 5)) / 25.0

# Appliquer le filtre moyennneur à l'image bruitée
image_filtered_mean = apply_filter(image_noisy, mean_kernel)

# Afficher les images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image_noisy, cmap='gray')
plt.title('Image Bruitée')

plt.subplot(1, 3, 2)
plt.imshow(image_filtered_gaussian, cmap='gray')
plt.title('Filtre Gaussien 7x7')

plt.subplot(1, 3, 3)
plt.imshow(image_filtered_mean, cmap='gray')
plt.title('Filtre Moyennneur 5x5')

plt.show()
```

